MOBILE DEVELOPMENT

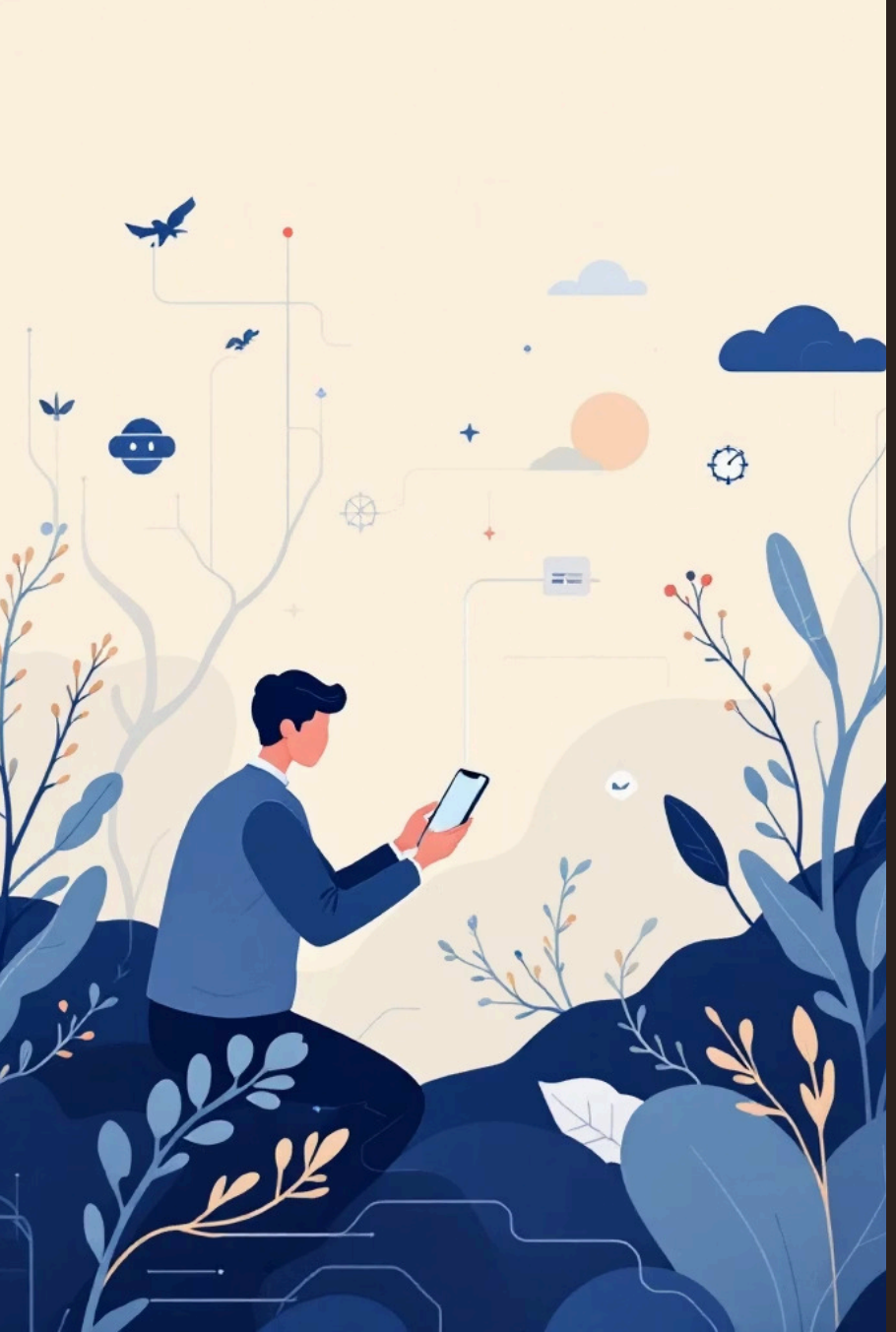
Android / iOS with Flutter

Renel Sainfort
ESILV – Engineering School

CM3: Flutter & Document System AI

SESSION

01 — Introduction
02 — Flutter Architecture
03 — The AI Documents System
04 — What's Next



Chapter 01

Introduction

What You Built Last Time

In TP2, you analyzed a screen, identified widgets (`Row`, `Column`, `Container`...), and wrote an optimized **RCTFC prompt** for a single screen:

1	2	3	4	5
R	C	T	F	C
Role	Context	Task	Format	Constraint

And if you had 5 screens to generate?

Problem 1 — Visual Inconsistency

🎨 Colors

📐 Spacing

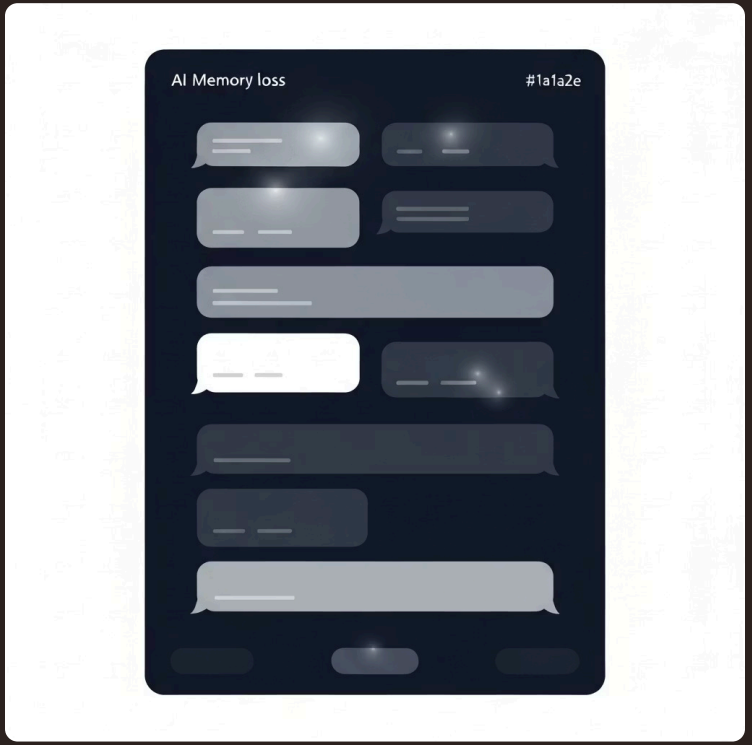
🔤 Typography

Problem 2 — Loss of Context

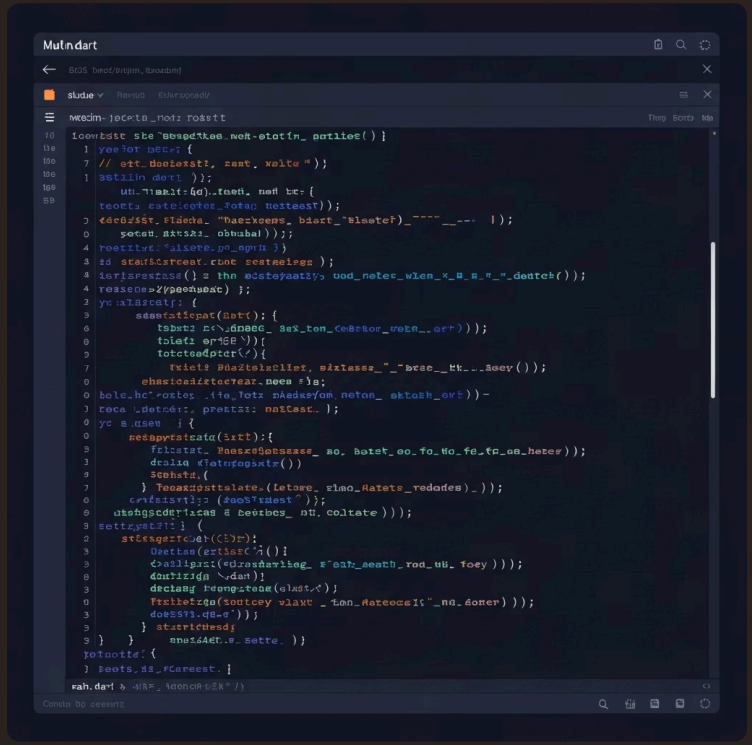
Repetition

Contradiction

Restart



Problem 3 — Code You Don't Control



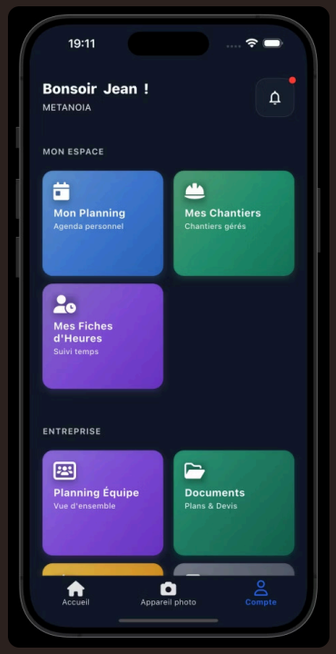
Unreadable

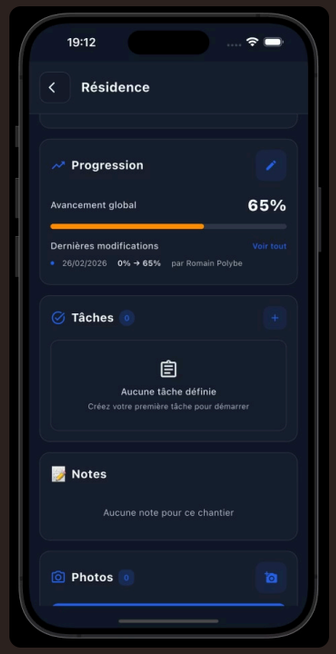
Insecure

Unpredictable

BatiNeoPro - 100% AI-Generated

A mobile-first SaaS platform for construction companies – managing worksites, teams, tasks and documents from the field.



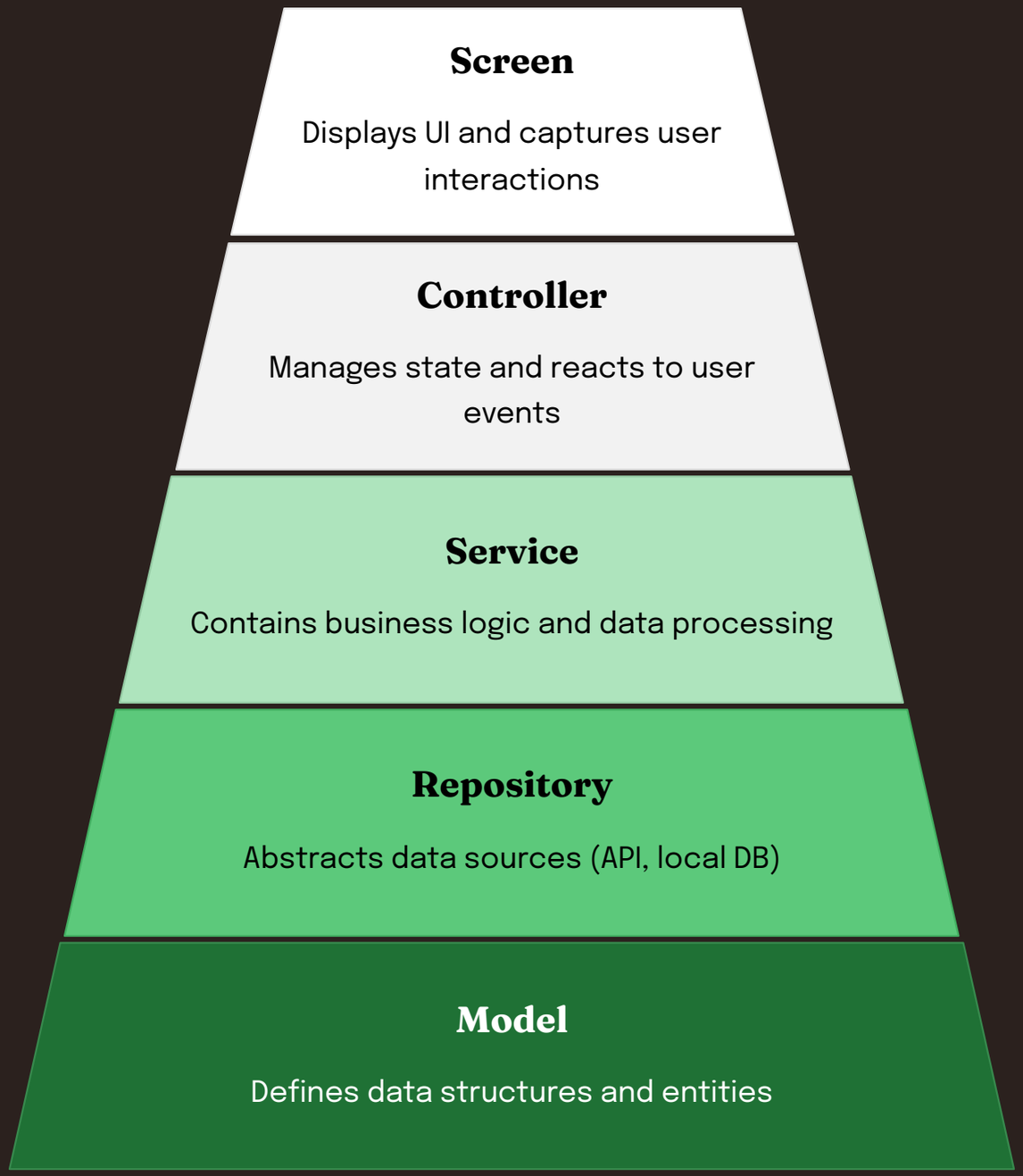


56	50	36	206	92K
Screens	Database Tables	Data Models	Dart Files	Lines of Code

Chapter 02

Flutter Architecture

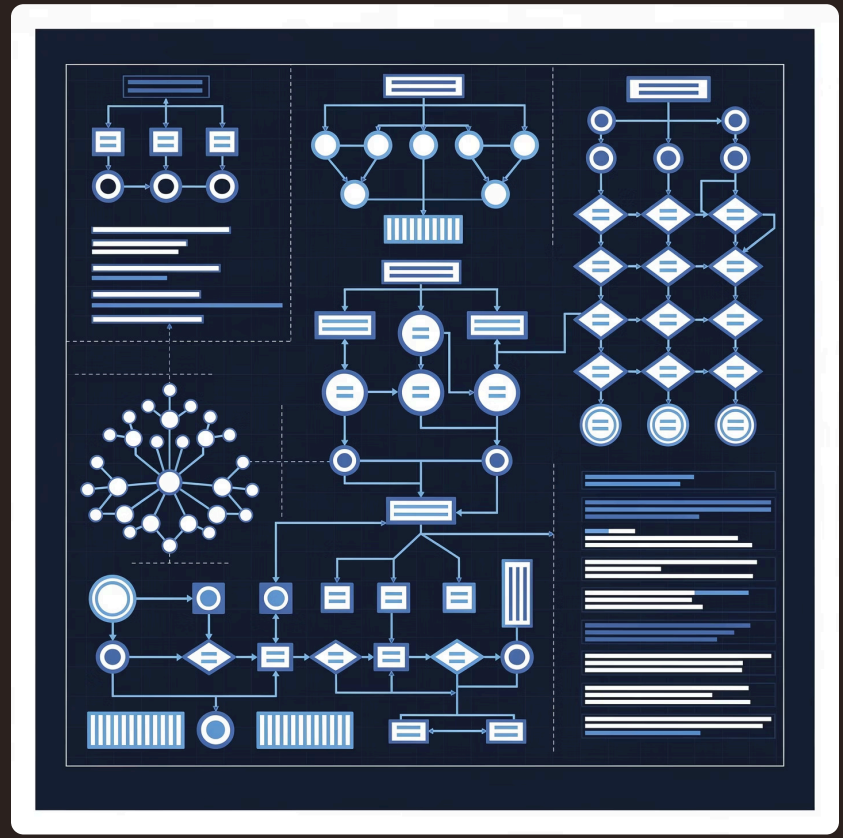
How a Flutter App is Built?



Data Storage

Model

The structure of your data before anything is built.



A Model describes what your data looks like – its fields, its types, its shape. Every object in your app (a user, a group, an expense) is defined as a Model.

Package

→ Freezed generates the translation between your database and your app – automatically, without errors.

- Immutable data
- Auto-generated

Repository

The only layer that talks to the outside world.

The Repository fetches and stores data – nothing more, nothing less. It's the only layer allowed to communicate with a database or an external API.

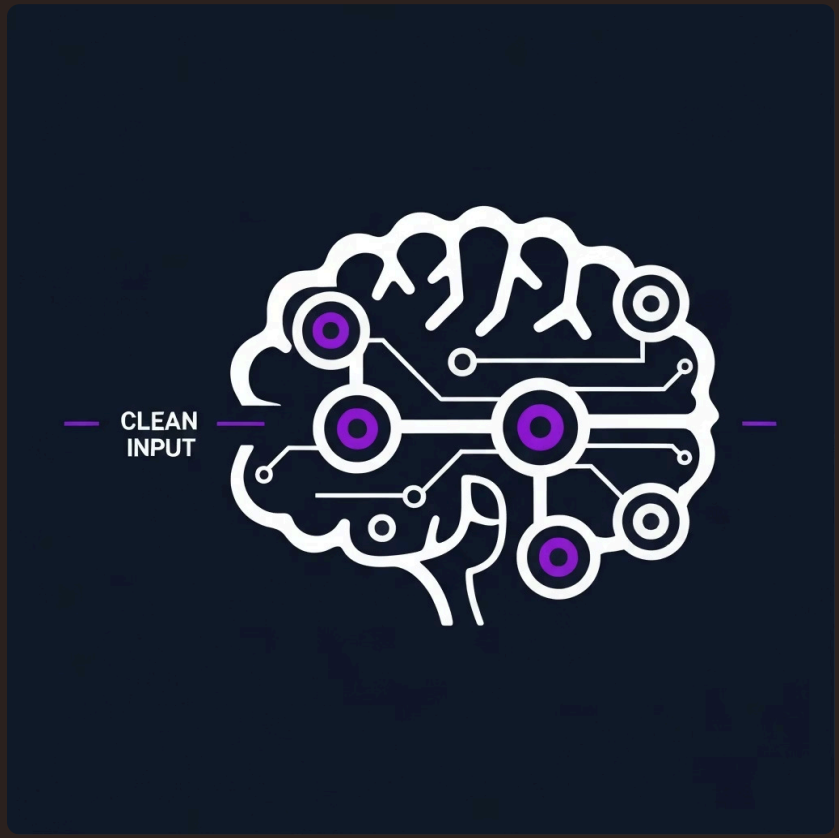
2 data sources :

→ **Local** – Hive, Isar, SQLite *Stored on the device. Works offline.*

→ **External** – Supabase, Firebase, REST API *Stored on a server. Requires internet.*

Service

Where the logic lives



The Screen displays the interface and captures user interactions. It never calculates, never stores – it only shows and delegates.

Key rule:

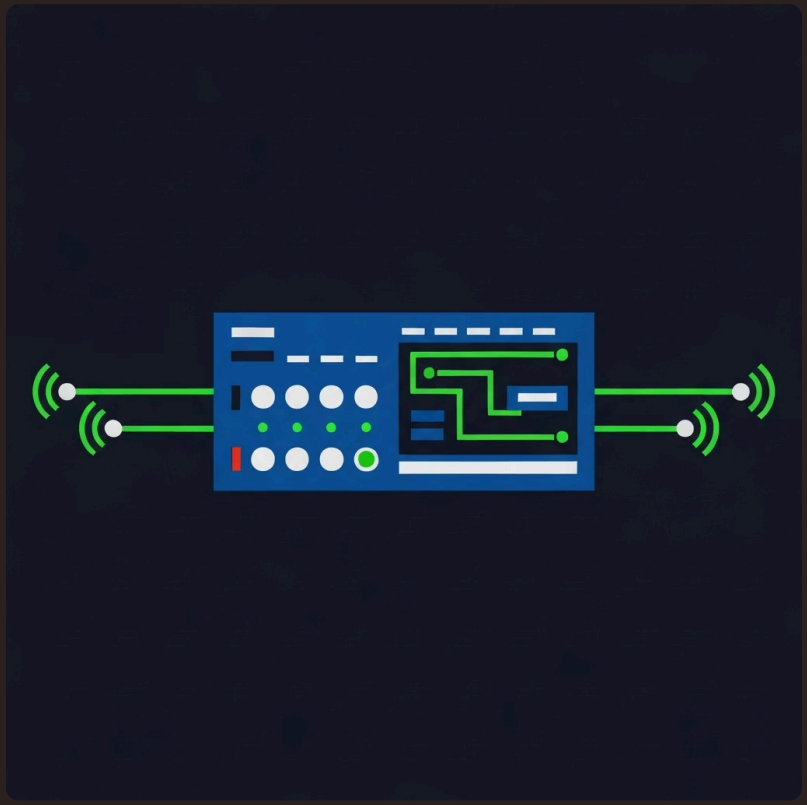
→ **No Flutter dependency** – pure Dart only

→ **No direct database calls** – always goes through the Repository

Controller

The bridge between logic and interface.

The Controller manages the state of your app. It listens to the Service and notifies the Screens when data changes.



Key rule :

→ **Riverpod** – StateNotifier or AsyncNotifier

→ **No business logic** – only state management

Screen

What the user sees and touches.

The Screen displays the interface and captures user interactions. It never calculates, never stores – it only shows and delegates.

Key rule:

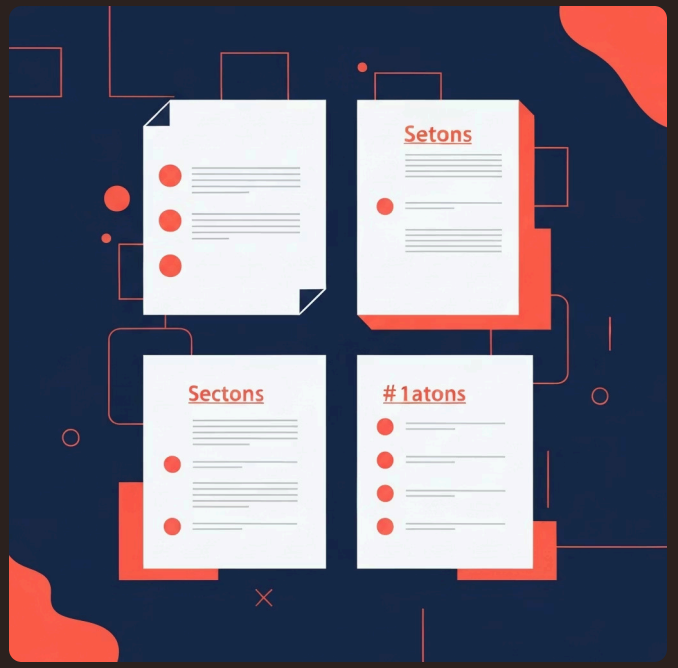
→ **ConsumerWidget** – connected to the Controller via Riverpod

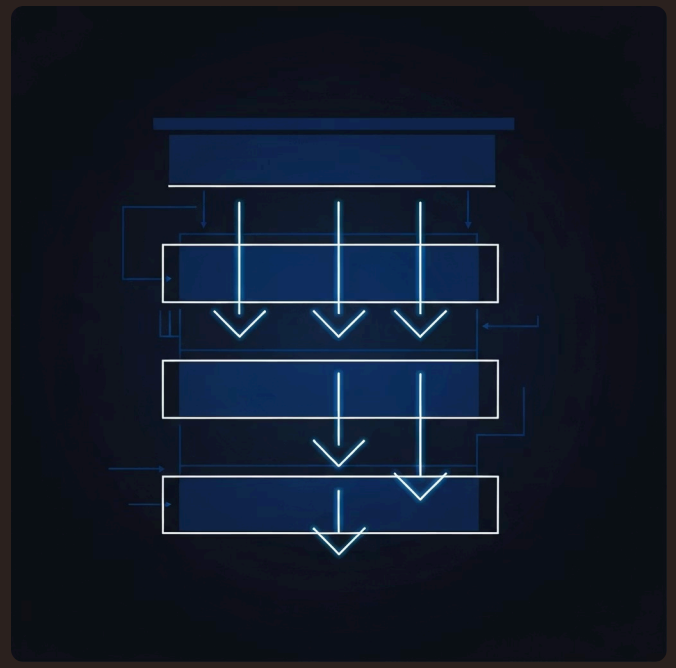
→ **No business logic** – every action is sent to the Controller

CHAPTER 03

The AI Documents System

The 3 Pillars of AI Development





Brief PO

What to build?

Architecture

How does it work?

Design System

How does it look?

Brief PO

The Brief PO is the first document to fill – before any code, before any design. It defines every screen, every action, and every data of your app.

#	Section	In one line
1	Concept	What is it, for who, what problem?
2	Screens	Screens, components, wireframes, 6 states
3	Actions & Filters	Every user action per screen
4	Data	Data source and Models
5	Out of scope	What we are NOT building
6	Navigation	How screens are connected
7	Success criteria	How we know it's done

Feeds which layers:

Service | **Controller** | **Screen**

Architecture

The Architecture is the second document to fill – after the Brief PO. It defines HOW to build what the Brief PO described.

#	Section	In one line
1	Project Structure	File tree, names, folders
2	Data Models	Every object, every field, every type
3	Who Does What	Repository, Service, Controller – one job each
4	Data Source	Where data comes from, where it's stored
5	Packages	What we install, nothing more

Feeds which layers:

Models | **Repository** | **Service**

Design System

The Design System is the third document to fill – after the Brief PO and Architecture. It defines every color, every text style, and every spacing of your app.

#	Section	In one line
1	Inspiration	One reference, one mood
2	Colors	Every color, every role
3	Typography	Every text style, size and weight
4	Spacing	Radius, padding, gaps

Feeds which layers:

Screen

The Pilot File

The Pilot File is the config file that every AI agent reads before generating code. It defines the AI's role, points to your 3 documents, and sets the rules.

AI Agent	Config file	Location
Claude Code	CLAUDE.md	root
GitHub Copilot	copilot-instructions.md	.github/
Gemini Code Assist	GEMINI.md	root
Cursor	.cursorrules	root

What goes inside:

#	Section	In one line
1	Your role	Who is the AI – senior Flutter dev
2	The project	What we're building – one sentence
3	The documents	Where to read – Brief PO, Architecture, Design System
4	The golden rule	Follow the docs, not your imagination
5	Constraints	English code, layer by layer, wait for validation

Your Project Structure

```
my_app/
├── docs/
│   ├── BRIEF_PO.md
│   ├── ARCHITECTURE.md
│   └── DESIGN_SYSTEM.md
├── CLAUDE.md          ← points to /docs/
├── GEMINI.md          ← points to /docs/
├── .github/
│   └── copilot-instructions.md  ← points to /docs/
├── .cursorrules       ← points to /docs/
└── lib/
    └── ...
```

📁 docs/

Your 3 documents. Always the same.

⚙️ Config file

One per AI agent. At root.

🔄 Switch agent

Change one file. Docs untouched.

WHAT'S NEXT

TP3

Launching an App Project

CM4

Connect (Database / API / MCP)